DL_LAB5

資工碩一

ID : 314551147

Name : 楊承恩

1. Introduction

This LAB consists of three tasks. Task1 uses Vanilla DQN to train CartPole, Task2 uses Vanilla DQN to train Atari, and Task3 applies Double DQN, PER, and Multi-Step Return to train Atari. In the class DQN main network, I implemented two versions: one for CartPole without convolutional layers, because CartPole's input is not a stack of images but a low-dimensional numerical vector, and another for Atari using convolutional layers since its input is a stack of images. The TD target for Task1 and Task2 is

Q(s,a) = r + γ(1−done)·max_{a'}Q_target(s',a')

while Task3, which uses Double DQN, applies

Q(s,a) = r+γ^n(1−done)Qtarget(s', argamaxQonline(s',a))

The final loss is computed using Huber loss. For the optimizer, I used RMSprop for improved stability. For epsilon, I adopted a linear decay schedule.

2. Your implementation

- **How do you obtain the Bellman error for DQN?**

```
with torch.no_grad():
    if self.use_double_dqn:
        a_star = self.q_net(next_states).argmax(dim=1, keepdim=True)
        next_q_max = self.target_net(next_states).gather(1, a_star).squeeze(1)
    else:
        next_q_max = self.target_net(next_states).max(dim=1).values

    targets = rewards + gammas * (1.0 - dones) * next_q_max  # r + γ(1-done)max_a' Q_target(s',a')

td_error = q_values - targets
```

Vanilla DQN :

Use y = r + γ(1−done)·max_{a'}Q_target(s',a') and then compute Qactual−y

Double DQN：

Use y = r+γ^n(1−done)Qtarget(s', argamaxQonline(s',a)) and then compute Qactual−y

- ## How do you modify DQN to Double DQN?

The main change is replacing the TD error calculation with the Double DQN method.

I use an args parameter use_double_dqn to determine whether to apply Double DQN.

If using DQN: Qactual– (r + γ(1−done)·max_{a'}Q_target(s',a'))

If using Double DQN: Qactual–(r+γ^n(1−done)Qtarget(s', argamaxQonline (s',a)))

- ## How do you implement the memory buffer for PER?

add(self, transition, error): Stores a transition in the buffer and assigns its priority, calculated as abs(error) + eps. The priorities are stored in a priorities array, where error refers to the TD error.

sample(self, batch_size, beta=None): Samples a batch according to the priorities. First, the priorities are converted into a probability distribution, then a batch is drawn based on these probabilities. Finally, the IS (importance sampling) weights are computed.

update_priorities(self, indices, errors): Updates the priorities based on the latest TD errors.

```python
class PrioritizedReplayBuffer:
    """
        Prioritizing the samples in the replay memory by the Bellman error
        See the paper (Schaul et al., 2016) at https://arxiv.org/abs/1511.05952
    """
    def __init__(self, capacity, alpha=0.6, beta=0.4, eps=1e-6):
        self.capacity = capacity
        self.alpha = alpha
        self.beta = beta
        self.eps = eps
        self.buffer = [None] * capacity
        self.priorities = np.zeros((capacity,), dtype=np.float32)
        self.pos = 0
        self.size = 0

    def __len__(self):
        return self.size

    def maxlen(self):
        return self.capacity

    def add(self, transition, error):
        if error is None:
            max_prio = self.priorities.max() if self.size > 0 else 1.0
        else:
            max_prio = abs(error) + self.eps
        self.buffer[self.pos] = transition
        self.priorities[self.pos] = max_prio
        self.pos = (self.pos + 1) % self.capacity
        self.size = min(self.size + 1, self.capacity)

    def sample(self, batch_size, beta=None):
        if beta is None:
            beta = self.beta

        prios = self.priorities[:self.size]
        probs = prios ** self.alpha
        probs = probs / probs.sum()

        indices = np.random.choice(self.size, batch_size, p=probs, replace=False)
        samples = [self.buffer[i] for i in indices]

        N = self.size
        weights = (N * probs[indices]) ** (-beta)
        weights = weights / weights.max()
        return indices, samples, weights.astype(np.float32)

    def update_priorities(self, indices, errors):
        for idx, err in zip(indices, errors):
            self.priorities[idx] = abs(float(err)) + self.eps
```

- ## How do you modify the 1-step return to multi-step return?

get_n_step_transition(): Responsible for accumulating the rewards over nnn steps into a single n-step transition. In simple terms, it calculates the reward over nnn steps.

$$R_n = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots + \gamma^{n-1} r_{t+n-1}$$

store_transition(self, transition): Used to add a transition to memory.

In the second image, n_step > 1 means multi-step return is being used. Each step's information is first pushed into n_step_buffer.

```python
def get_n_step_transition(self):
    total_reward = 0.0
    gamma = 1.0
    for (_, _, reward, _, done) in self.n_step_buffer:
        total_reward += gamma * reward
        gamma *= self.gamma
        if done:
            break
    state_first, action_first, _, _, _ = self.n_step_buffer[0]
    _, _, _, next_state_last, done_last = self.n_step_buffer[-1]
    return (state_first, action_first, total_reward, next_state_last, done_last, gamma)

def store_transition(self, transition):
    if self.use_per:
        self.memory.add(transition, error=None)
    else:
        self.memory.append(transition)
```

```python
if self.n_step > 1:
    self.n_step_buffer.append((state, action, reward, next_state, done))
    if len(self.n_step_buffer) == self.n_step or done:
        s0, a0, Rn, sN, dN, gamma_n = self.get_n_step_transition()
        self.store_transition((s0, a0, Rn, sN, dN, gamma_n))
        if not done:
            self.n_step_buffer.popleft()
else:
    self.store_transition((state, action, reward, next_state, done, self.gamma))
```

When nnn steps have been accumulated or the episode ends early, get_n_step_transition() is called to generate an n-step transition. This transition is then stored in the replay buffer and later used during training to directly compute the TD target.

- **Explain how you use Weight & Bias to track the model performance**

```python
if self.env_count % 1000 == 0:
    print(f"[Collect] Ep: {ep} Step: {step_count} SC: {self.env_count} UC: {self.train_count} Eps: {self.epsilon:.4f}")
    wandb.log({
        "Episode": ep,
        "Step Count": step_count,
        "Env Step Count": self.env_count,
        "Update Count": self.train_count,
        "Epsilon": self.epsilon
    })
```

```python
print(f"[Eval] Ep: {ep} Total Reward: {total_reward} SC: {self.env_count} UC: {self.train_count} Eps: {self.epsilon:.4f}")
wandb.log({
    "Episode": ep,
    "Total Reward": total_reward,
    "Env Step Count": self.env_count,
    "Update Count": self.train_count,
    "Epsilon": self.epsilon
})
```

```python
wandb.log({
    "Env Step Count": self.env_count,
    "Update Count": self.train_count,
    "Eval Reward": eval_reward
})
```

```python
if self.train_count % 1000 == 0:
    wandb.log({
        "Loss": loss.item(),
        "buffer_len": self.mem_len(),
        "use_per": int(self.use_per),
        "use_ddqn": int(self.use_double_dqn),
        "n_step": self.n_step,
        "per_beta": getattr(self, "per_beta", 0.0),
        "Env Step Count": self.env_count,
        "Update Count": self.train_count,
    })
```

Training process

Loss: Current batch's Huber Loss value

buffer_len: Number of transitions currently stored in the Replay Buffer

use_per: Whether Prioritized Experience Replay is enabled (0 = no, 1 = yes)

use_ddqn: Whether Double DQN is enabled (0 = no, 1 = yes)

n_step: Number of steps for the n-step return

per_beta: β value for importance sampling in PER

Env Step Count: Total number of interactions with the environment (env_count)

Update Count: Number of network parameter updates (train_count)

Evaluation process

Env Step Count: Total number of interactions with the environment

Update Count: Number of network updates

Eval Reward: Average return obtained during evaluation (eval_reward)

At the end of an episode

Episode: Current episode number

Total Reward: Total return of the episode

Env Step Count: Total number of interactions with the environment

Update Count: Number of updates

Epsilon: Current ε-greedy exploration rate

Data collection process

Episode: Current episode number

Step Count: Number of steps taken in the current episode

Env Step Count: Total number of interactions with the environment

Update Count: Number of updates

Epsilon: Current ε-greedy exploration rate

3. Analysis and discussions

Task1:

Task2:

Task3:

Training Curves:

Task 2:

The Eval Reward remained near -20 for an extended period in the early stage, with occasional small fluctuations. It was not until around step 1500 that a more stable improvement appeared, eventually converging to about -13. The Total Reward showed a similar trend, with rewards staying in the negative range for most of the training process, indicating a slower learning speed.

Task 3:

The Eval Reward began to rise significantly after around 200k steps and eventually surpassed 0 in the later stage, stabilizing between 5 and 10. The Total Reward also showed a steady upward trend, maintaining mostly positive values in the second half of training, suggesting that the model had learned an effective strategy.

Sample Efficiency Analysis:

In comparison, Task 3 achieved positive rewards with relatively fewer training samples, indicating that DQN enhancements (such as Double DQN, Dueling Network, and Prioritized Experience Replay) effectively improved sample efficiency. Task 2, on the other hand, required a much longer training time to escape the low-reward range, showing significantly lower learning efficiency.

Wandb:

Task1: https://wandb.ai/jason052102-national-yang-ming-chiao-tung-university/DLP-Lab5-DQN-CartPole/runs/h6d3nfz8

Task2: https://wandb.ai/jason052102-national-yang-ming-chiao-tung-university/DLP-Lab5-

DQN-ALE/runs/eay0lcyv

Task3: https://wandb.ai/jason052102-national-yang-ming-chiao-tung-university/DLP-Lab5-DQN-ALE/runs/oucka8jz

Run Task1:

python dqn.py --wandb-run-name cartpole-run --replay-start-size 1000 --target-update-frequency 500 --epsilon-decay 0.9995 --lr 0.001 --batch-size 64 --discount-factor 0.98

Run Task2:

python dqn.py --wandb-run-name ale-run --memory-size 100000 \--replay-start-size 50000 --target-update-frequency 10000 --batch-size 32 \--epsilon-start 1.0 --epsilon-min 0.1 --epsilon-decay-steps 1000000 --n-step 1

Run Task3:

python dqn.py --wandb-run-name ale-run --use-double-dqn --use-per --n-step 3--memory-size 100000 --replay-start-size 50000 --target-update-frequency 10000 --batch-size 32 --epsilon-start 1.0 --epsilon-min 0.1 --epsilon-decay-steps 1000000 --per-alpha 0.6 --per-beta0 0.4 --per-beta-anneal-steps 1000000